

cobot_magic详细使用说明文档

[Cobot Magic-github地址](#)

- **jetson-orin-nano版本:** 只支持机械臂遥操作、采集数据、不支持模型训练、不支持模型推理
- **工控机(4090显卡版,4060显卡版):** 支持机械臂遥操作、采集数据、支持模型训练、支持模型推理
- ACT模型训练建议使用**高算力大显存服务器**, 4060显卡训练 50 episode、500 timesteps的数据集,batch-size给4都会出现显存不足。4060可用于推理
- **遇到问题请先参考第5(Q&A)章节**
- **出厂时已经配置好了软件环境, 拿到设备后只需从第三节“采集数据”开始操作**

1 上电配置

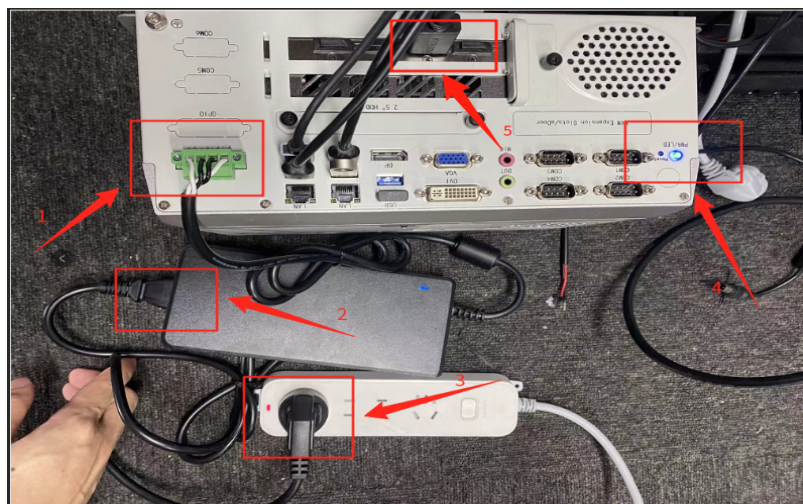
1.1 工控机上电配置

1.1.1 工控机上电

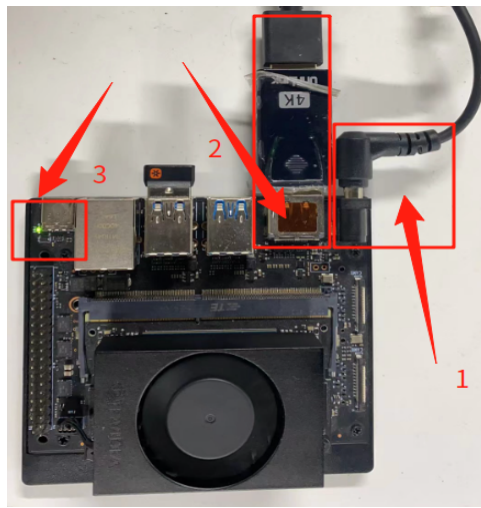
-工控机开机密码为: agx

-路由器WiFi名称: aloha 密码: 12345678

- 工控机上电如下图所示, 按图中箭头1、2、3接通电源, 通电后工控机会自动开机,箭头4处开机指示灯会亮起即可。
- 如果电脑关机后未断开电源,需要重启工控机按图中箭头4处开机按钮即可。
- 显示器输出HDMI(DP)线请插入显卡上的HDMI或者DP接口处, 如图中5所示。



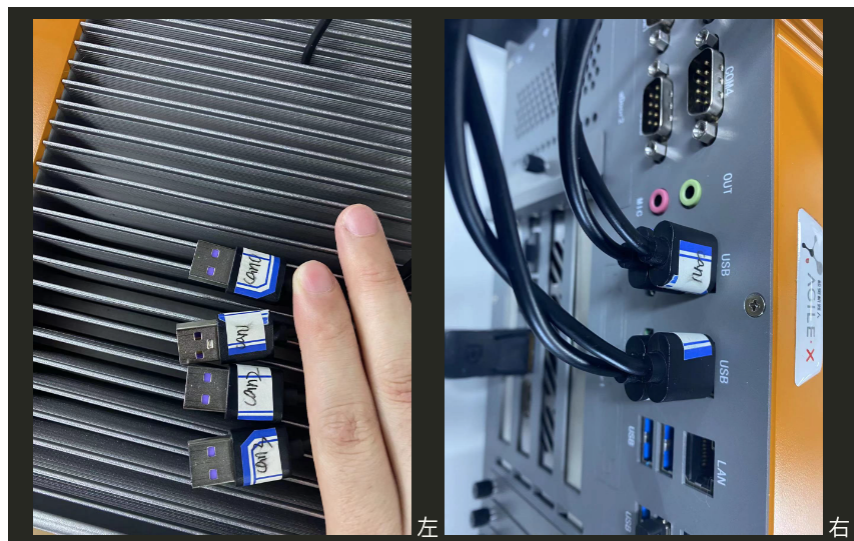
1.1.2 jetson-orin-nano上电



如上图所示,2处连接DP线供显示器输出, 1处接通电源,orin-nano会自动开机,3处开机指示灯会亮起即可

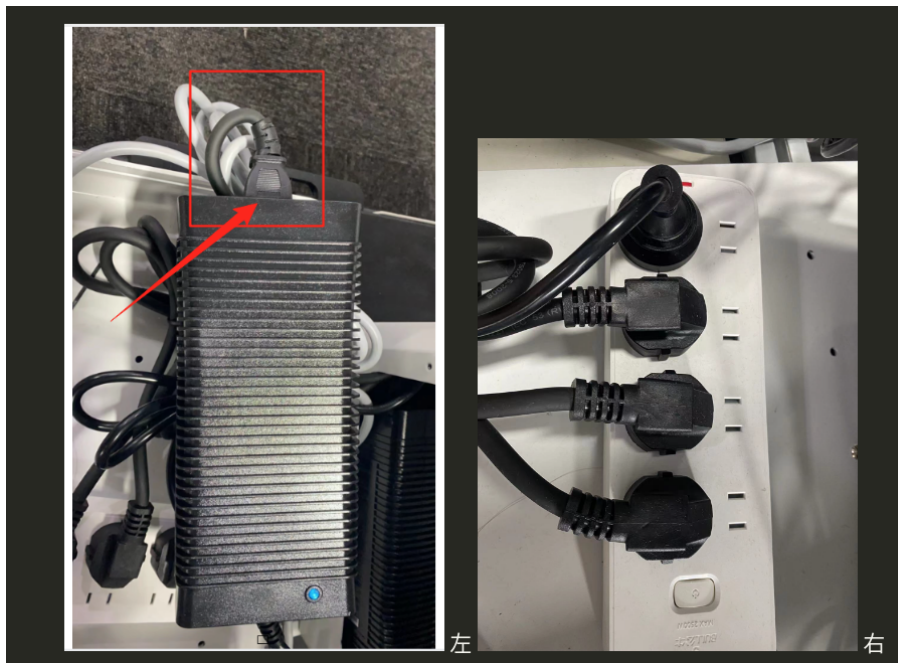
1.2 机械臂上电配置

1.2.1 机械臂接线



如上图所示,机械臂通过can转usb与工控机通讯, 这个usb接口不可随机插, 是预先定义的usb端口。

1.2.2 机械臂上电



如上图所示,分别按左图所示,接上适配器的插头;然后按右图所示,4条机械臂的适配器插头接上电源即可。

接通电源后,如下图机械臂有4处指示灯亮即表示通电成功,如果机械臂base的灯没有闪烁绿灯, 则检查机械臂供电或者机械臂的线材。

1.3 相机上电配置

1. 相机接线



如上图所示,左图分别是3个相机的3条usb线, 按右图所示分别接入工控机的usb口处(或者usb拓展坞上)即可。

2. 相机上电

相机供电由usb供电, 无需额外供电, 连接好usb线即可。

2 软件环境配置

- 此章节的操作出厂前已配置完毕, 无需重新配置。

- cobot_magic的工控机(4060显卡、4090显卡版本),镜像自带ubuntu-20.04、ros1-noetic、cuda-11.3、torch-1.10、conda、python-3.8, 镜像下载地址
 - jetson-orin-nano板载,镜像自带ubuntu-20.04、ros1-noetic
1. 数据采集基础配置：ubuntu-20.04、ros1-noetic
 2. 模型训练推理配置：1. ubuntu-20.04、ros1-noetic、cuda-11.3、torch-1.10(cuda-11.8、torch-2.1.1), conda,python-3.8, 已测试通过，cuda、torch其他版本请用户自己测试

- 下载cobot_magic源码

```
# 1 下载源码
git clone

# 2 进入cobot_magic工作目录
cd cobot_magic
```

2.1 相机配置

- 相机参数

Baseline	40mm
深度距离	0.3-3m
深度图分辨率	640x400x30fps、320x200x30fps
彩色图分辨率	1920x1080x30fps、1280x720x30fps、640x480x30fps
精度	6mm@1m (81%FOV区域参与精度计算)
深度FOV	H 67.9° V 45.3°
彩色FOV	H 71° V 43.7° @ 1920x1080
延迟	30-45ms
数据传输	USB2.0或以上
工作温度	10°C~40°C
尺寸	长59.5x宽17.4x厚11.1 mm

2.1.1 编译相机驱动

```
# 1 进入ros_astra_camera目录
cd camera_ws

# 2 安装依赖
```

```

sudo apt install libgflags-dev libgoogle-glog-dev libusb-1.0-0-dev libeigen3-dev
ros-$ROS_DISTRO-image-geometry ros-$ROS_DISTRO-camera-info-manager
ros-$ROS_DISTRO-image-transport ros-$ROS_DISTRO-image-publisher ros-$ROS_DISTRO-
libuvc-ros -y

# 3 编译
catkin_make

# 4 Install udev rules.
source devel/setup.bash && rospack list
roscd astra_camera
./scripts/create_udev_rules
sudo udevadm control --reload && sudo udevadm trigger

# 5 增加永久ros_astra_camera包的环境变量
## 5.1 进入astra_ws目录
cd ../..
## 5.2 永久增加环境变量
echo "source $(pwd)/devel/setup.bash" >> ~/.bashrc
## 5.3 环境变量生效
source ~/.bashrc

```

2.1.2 运行

工控机接上astra相机的usb

```

# 1 检测相机序列号
roslaunch astra_camera list_devices_node

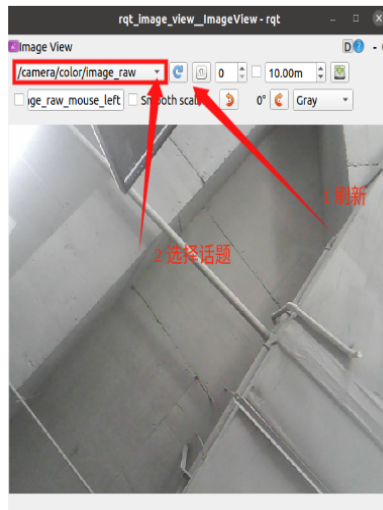
# 终端显示
## 1.1 显示如下： 表示相机未连接,请重新拔插usb
# [ INFO] [1710298393.524932942]: Found 0 devices

## 1.2 显示如下： 一个Serial number,即表示相机已经成功连接
# [ INFO] [1710298585.382430351]: Found 1 devices
# Device connected: Astra
# URI: 2bc5/0657@1/11
# Serial number: CC1GA35002L

# 2 启动相机
roslaunch astra_camera dabai.launch

# 3 打开rqt_image_view查看图像
rqt_image_view
## 显示如下图，注意先刷新再选择对应的话题即可显示图像

```

2.1.3 重新编译

```
catkin_make clean && catkin_make
```

2.1.4 配置多相机

- 这里测试3个相机, 工控机接上3个astra相机的usb

```
# 1 检测相机序列号
## 1.1 相机序列号查询,方法1
roslaunch astra_camera list_devices_node
## 终端显示如下:
# [ INFO] [1710299519.592966536]: Found 3 devices
# Device connected: Astra
# URI: 2bc5/060e@1/29
# Serial number: AU1231201GE
# Device connected: Astra
# URI: 2bc5/060e@1/35
# Serial number: AU1953304F2
# Device connected: Astra
# URI: 2bc5/060e@1/24
# Serial number: AU1P32201SA

## 发现3个devices, 即表示相机连接成功, 记住这3个Serial number, 后续会使用
## 未发现3个devices, 请检测相机usb线是否成功连接

## 1.2 相机序列号查询,方法2
## cobot_magic/tools目录下camera_serial.sh脚本, 注意路径,如果没有该文件就使用方法1
## 1.2.1 进入cobot_magic目录
cd cobot_magic
## 1.2.2 运行该脚本
./tools/camera_serial.sh

# 2 修改multi_camera.launch配置文件
## 修改camera_ws/src/ros_astra_camera/launch/multi_camera.launch第13左右相机的
Serial number值
## 将1中输出的3个Serial number值,分别填入下面代码中即可
<arg name="camera1_serila_number" default="AU1231201GE"/>
<arg name="camera2_serila_number" default="AU1953304F2"/>
<arg name="camera3_serila_number" default="AU1P32201SA"/>
```

```
# 3 启动多相机
roslaunch astra_camera multi_camera.launch
## 使用rqt_image_view即可查看相应的图像话题
```

2.2 机械臂配置

ps:出厂时，工控机已经配置好机械臂运行的环境。用户使用时，只需按照2.2.2命令启动机械臂即可。

2.2.1 环境安装

1. 安装依赖

```
sudo apt install can-utils net-tools libkdl-parser-dev ethtool -y
```

2. 机械臂can模块配置(首次配置)

agx四条机械臂总共2个usb转can模块，需要按次序来配置

在 `can_config.sh` 中，`EXPECTED_CAN_COUNT` 参数一般设置为 3，因为四条机械臂使用两个can模块，底盘使用一个can模块。

机械臂按顺序依次进行以下操作：

- 机械臂上电
- 机械臂左侧usb to can插入工控机(注意另一个不要插)
- 查看can路由是否被系统识别并查看can编号，一般是can0
- 读取当前can(如上读取的编号，一般是can0)模块所在的端口
- 机械臂右侧usb to can插入工控机(注意另一个不要拔)
- 查看can路由是否被系统识别并查看can编号，一般是can1
- 读取当前can模块(如上读取的编号，一般是can1)所在的端口

```
# 1 查看can模块路由
ip link show | grep can
## 终端显示can0，下面就用can0查看usb端口号
## 插上机械臂can模块才会有终端显示，如果终端没有输出，请重新拔插

# 2 查看can模块所在的usb端口号，记录下`bus-info`的数值例如`1-2:1.0`
sudo ethtool -i can0 | grep bus
sudo ethtool -i can1 | grep bus
```

将左侧臂和右侧臂的can模块都插入到电脑并执行了如上指令后，记录下 `bus-info` 的数值

假设can0(左侧)对应的usb编号为 `1-2:1.0`，can1(右侧)对应的usb编号为 `1-4:1.0`

将 `can_config.sh` 的 `USB_PORTS["1-9:1.0"]="can_left:1000000"` 中的中括号内部的双引号内部的参数换为 `1-2:1.0`

```
USB_PORTS["1-9:1.0"]="can_right:1000000" -> USB_PORTS["1-2:1.0"]="can_left:1000000"
```

另一个同理

```
USB_PORTS["1-5:1.0"]="can_right:1000000" -> USB_PORTS["1-4:1.0"]="can_right:1000000"
```

最后的参数如下

```
if [ "$EXPECTED_CAN_COUNT" -ne 1 ]; then
    declare -A USB_PORTS
    USB_PORTS["1-2:1.0"]="can_left:1000000"
    USB_PORTS["1-4:1.0"]="can_right:1000000"
    USB_PORTS["1-3:1.0"]="can0:500000"
fi
```

ps:此处的意思是，1-2:1.0硬件编码的usb端口插入的can设备，名字被重命名为can_left，波特率为1000000，并激活

- 执行 `bash can_config.sh` 输入密码后查看两个can设备是不是激活成功
- 执行 `ifconfig | grep can` 查看是不是有 `can_left` 和 `can_right`，如果有则can模块设置成功

注意：

每次开机后，或者每次拔插can模块后，都需要执行“`bash can_config.sh`”

如果更换了usb端口，请重复上述操作

2.2.2 仅获取主从机械臂关节消息（采集数据）

下面的参数含义：

`can_port`：为指定打开的can端口名字

`mode`：为设定打开模式，

- 1，则发送主臂和从臂的关节信息，而不注册控制从臂的回调函数
- 0，则只发送从臂关节信息而不发送主臂的，并且注册从臂的控制回调函数

```
cd cobot_magic/agx_arm/
source devel/setup.bash
roslaunch agx_arm start_all_agx_arm.launch mode:=1
或使用以下命令：
roslaunch agx_arm agx_arm_start_all_left_node.py _can_port:="can_left" _mode:=1
roslaunch agx_arm agx_arm_start_all_right_node.py _can_port:="can_right" _mode:=1
```

然后执行

```
rostopic list
```

有如下几个topic：

```
/arm_status
/enable_flag
/joint_feedback
/master/joint_left
/master/joint_right
/puppet/joint_left
/puppet/joint_right
```

其

中 `/master/joint_left`、`/master/joint_right`、`/puppet/joint_left`、`/puppet/joint_right` 可以读取到主臂和从臂的消息

通过节点控制从臂（执行重播数据，推理）

先将主从臂的数据线断开，再执行以下指令：

```
roslaunch agx_arm start_all_agx_arm.launch mode:=0
或使用以下命令：
roslaunch agx_arm agx_arm_start_all_left_node.py _can_port:="can_left" _mode:=0
roslaunch agx_arm agx_arm_start_all_right_node.py _can_port:="can_right" _mode:=0
```

然后执行

```
rostopic list
```

有如下几个topic：

```
/arm_status
/enable_flag
/joint_feedback
/master/joint_left
/master/joint_right
/puppet/joint_left
/puppet/joint_right
```

进行重播数据集和推理前，要给 `/enable_flag` topic 发送true，令其使能；

给enable_flag发送true会同时使能两个从臂。

```
rostopic pub /enable_flag std_msgs/Bool "data: true"
```

其中，`/master/joint_left`、`/master/joint_right` 是用来外部控制的topic，进而控制从臂

ps:如果中断程序后再打开节点，如果中断程序时机械臂的状态为使能，那么在中断程序后机械臂仍旧会保持使能，然后再重新运行节点，如果想要机械臂运动，需要重新给 `/enable_flag` 发送true

注意事项

- 执行执行重播数据，推理，需要先将主从臂的数据线断开!!!
- 在节点参数mode为1的情况
下 `/master/joint_left`、`/master/joint_right`、`/puppet/joint_left`、`/puppet/joint_right` 仅仅为数据topic，只需要读取，一般这种模式下，主臂和从臂的can是连通的
- 在节点参数mode为0的情况下，`/master/joint_left`、`/master/joint_right` 为控制从臂的topic，`/puppet/joint_left`、`/puppet/joint_right` 为从臂反馈，想要给 `/master/joint_left`、`/master/joint_right` 发送数据运动从臂之前，必须要给 `/enable_flag` topic 发送true

3 数据采集

- 默认ubuntu20.04-noetic环境已经配置完成。

3.1 环境依赖

```
# 1 ros依赖
sudo apt install ros-$ROS_DISTRO-sensor-msgs ros-$ROS_DISTRO-nav-msgs
ros-$ROS_DISTRO-cv-bridge

# 2 python依赖
## 2.1 激活python环境
conda activate aloha

## 2.2 安装torch
# torch==2.1.1
# pip install torch==2.1.1 torchvision==0.16.1 torchaudio==2.1.1 --index-url
https://download.pytorch.org/whl/cu118

# torch==1.10.0
pip install torch==1.10.0+cu113 torchvision==0.11.0+cu113 torchaudio==0.10.0 --
extra-index-url https://download.pytorch.org/whl/cu113

## 2.3 进入当前工作空间目录,安装requirements.txt文件中的依赖即可
cd collect_data

## 2.4 安装requirements.txt
pip install -r requirements.txt

## 2.5 安装detr
cd act/detr && pip install -v -e .
```

3.2 运行

3.2.1 采集数据

0.硬件检查

- 采集数据前需要检查四条机械臂的数据线是否插好，且亮绿灯



1. 启动机械臂、相机

- 启动硬件前, 请检查机械臂电源、通讯线, 相机通讯线是否成功连接
- 如果之前已经启动了机械臂遥操作代码, 这里就不需要启动机械臂遥操作, 只用启动相机即可

1 启动硬件

1.1 新建一个终端, 启动

```
roscore
```

1.2 进入agx_arm目录, 使能can, 并启动机械臂数据读取

```
cd cobot_magic/agx_arm/
```

```
bash can_config.sh
```

```
roslaunch agx_arm start_all_agx_arm.launch mode:=1
```

1.3 启动相机

```
roslaunch astra_camera multi_camera.launch
```

1.4 终端运行 `rostopic list` 查看 `ros` 话题

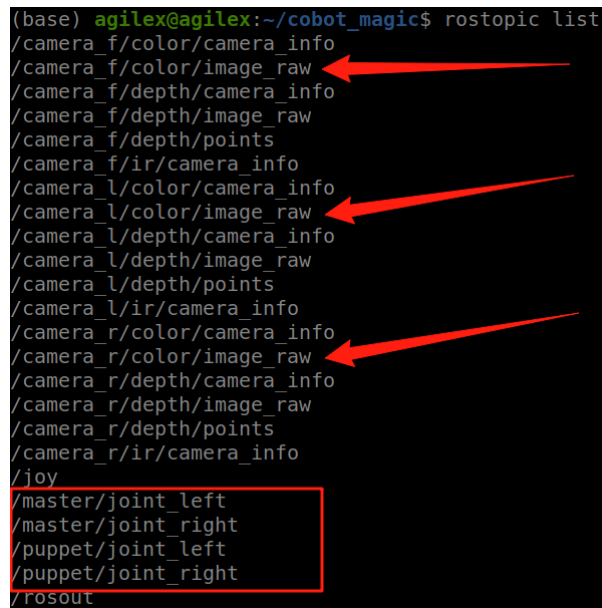
```
rostopic list
```

显示如下图，有机械臂4个、相机3个话题即可采集数据

1.5 一次打印上面 `ros` 话题的内容，保证采集数据前，传感器数据正常

```
rostopic echo 话题名
```

终端会按频率打印传感器数据，传感器有数据变化即可



```
(base) agilex@agilex:~/cobot_magic$ rostopic list
/camera_f/color/camera_info
/camera_f/color/image_raw
/camera_f/depth/camera_info
/camera_f/depth/image_raw
/camera_f/depth/points
/camera_f/ir/camera_info
/camera_l/color/camera_info
/camera_l/color/image_raw
/camera_l/depth/camera_info
/camera_l/depth/image_raw
/camera_l/depth/points
/camera_l/ir/camera_info
/camera_r/color/camera_info
/camera_r/color/image_raw
/camera_r/depth/camera_info
/camera_r/depth/image_raw
/camera_r/depth/points
/camera_r/ir/camera_info
/joy
/master/joint_left
/master/joint_right
/puppet/joint_left
/puppet/joint_right
/rosout
```

2. 采集数据

1 启动 `roscore`

```
roscore
```

2 激活虚拟环境

```
conda activate aloha
```

3 启动采集数据

3.1 进入 `collect_data` 目录

```
cd collect_data
```

3.2 查看 `collect_data.py` 配置参数

```
python collect_data.py -h # 查看参数
```

3.3 采集数据

```
python collect_data.py --dataset_dir ~/data --max_timesteps 500 --episode_idx 0
```

- 数据采集时终端显示如下图所示：

终端打印 `sync fail` 是正常的,表示当前时刻没同步传感器数据,只要终端不是一直 `sync fail` 而没有

`Frame data: xxx` 输出,都是正常现象

只要终端一直有 `Frame data: xxx` 打印信息,即表示正在记录数据集

如果打印 `sync fail`, 后续没有其他输出, 证明传感器数据没接收到, 请使用 `rostopic echo` 话题名 检查 `ros` 话题是否成功发布

```
(aloha) agilex@aloha-nano:~/cobot_magic/collect_data$ python collect_data.py --dataset_dir ~/data --max_timesteps 500 --episode_idx 0
syn fail
syn fail
Frame data: 2
Frame data: 3
syn fail
Frame data: 4
Frame data: 5
syn fail
Frame data: 6
Frame data: 7
syn fail
Frame data: 8
Frame data: 9
Frame data: 10
syn fail
Frame data: 11
Frame data: 12
syn fail
Frame data: 13
Frame data: 14
syn fail
Frame data: 15
```

- 数据保存路径说明

数据采集完成后会保存到`\${dataset\_dir}/\${task\_name}`目录下
运行上面这句代码, 其他参数采用默认值。产生数据集 `episode_0.hdf5`, 该工程目录如下

```
collect_data
├── collect_data.py
├── data                                # --dataset_dir 数据集保存路径
│   ├── aloha_mobile_dummy            # --task_name 任务名
│   │   ├── episode_0.hdf5            # 产生数据集文件的位置
│   │   ├── episode_idx.hdf5          # idx由--episode_idx参数决定
│   │   └── ...
├── readme.md
├── replay_data.py
├── requirements.txt
└── visualize_episodes.py
```

- `collect_data.py` 参数详细介绍

ps: 本代码暂时不支持深度图采集, 代码已开源, 可自行修改源码适配深度图采集

<code>--dataset_dir</code>	数据集保存路径
<code>--task_name</code>	任务名, 作为数据集的文件名
<code>--episode_idx</code>	动作分块索引号
<code>--max_timesteps</code>	最大动作分块的时间步数
<code>--camera_names</code>	相机名称, 默认 ['cam_high', 'cam_left_wrist', 'cam_right_wrist']
<code>--img_front_topic</code>	相机1彩色图话题
<code>--img_left_topic</code>	相机2彩色图话题
<code>--img_right_topic</code>	相机3彩色图话题
<code>--use_depth_image</code>	是否使用深度信息 # 本代码暂时不支持深度图采集, 代码已开源, 可自行修改源码适配
<code>--depth_front_topic</code>	相机1深度图话题
<code>--depth_left_topic</code>	相机2深度图话题

```
--depth_right_topic 相机3深度图话题

--master_arm_left_topic 左主臂话题
--master_arm_right_topic 右主臂话题
--puppet_arm_left_topic 左从臂话题
--puppet_arm_right_topic 右从臂话题
--use_robot_base      是否使用底盘信息
--robot_base_topic     底盘话题
--frame_rate           采集帧率 因相机图像稳定值为30帧,默认30帧
```

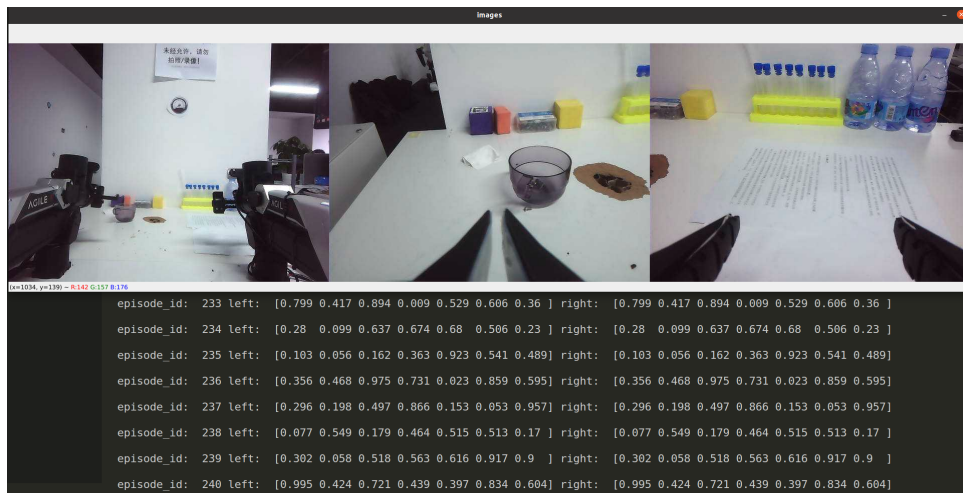
3.2.2 可视化数据集

3.2.2.1 运行

```
# 1 激活虚拟环境
conda activate aloha

# 2 运行visualize_episodes.py
python visualize_episodes.py --dataset_dir ~/data --task_name aloha_mobile_dummy
--episode_idx 0
```

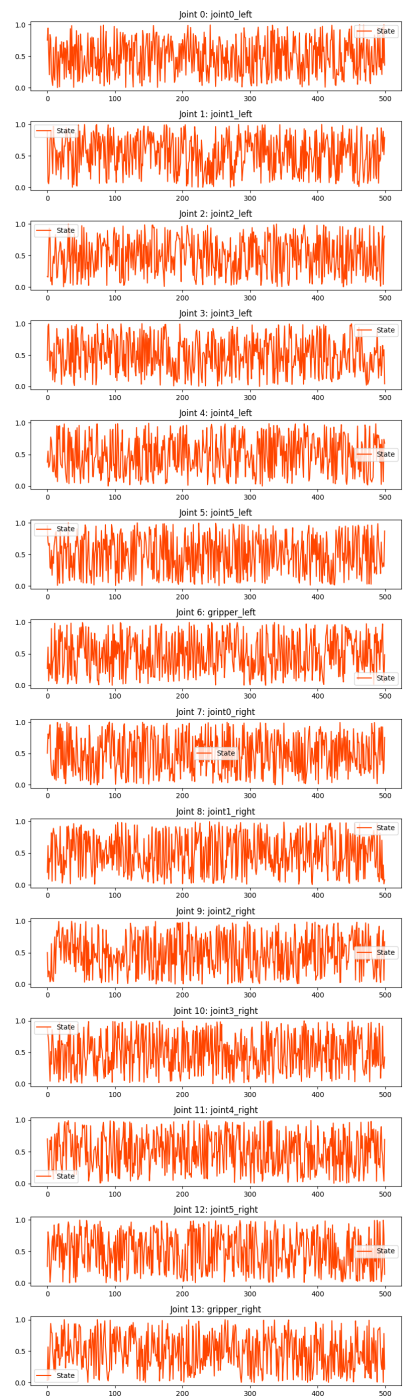
- 将3.2.1小节采集数据进行可视化运行上面代码。--dataset_dir、--task_name与--episode_idx参数需要与3.2.1小节采集数据时相同
- 运行上面代码,可视化结果如下:
终端会打印action,并显示一个彩色图像窗口



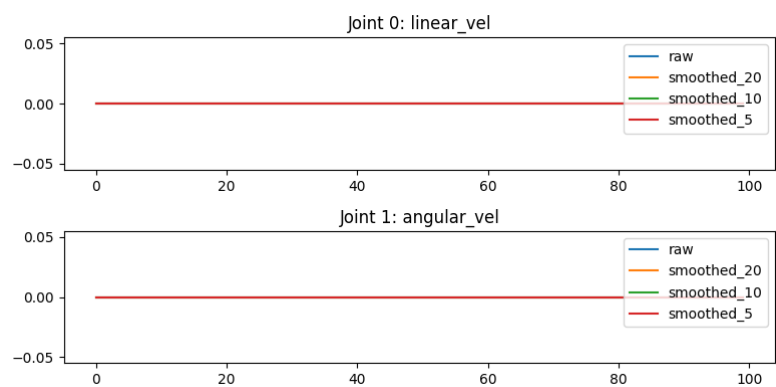
- 运行完成后,会在 \${dataset_dir}/\${task_name} 下产生 episode_\${idx}_qpos.png、episode_\${idx}_base_action.png 与 episode_\${idx}_video.mp4 文件,目录结构如下:

```
collect_data
├── data
│   ├── aloha_mobile_dummy
│   │   ├── episode_0.hdf5
│   │   ├── episode_0_base_action.png # base_action图
│   │   ├── episode_0_qpos.png       # qpos图
│   │   └── episode_0_video.mp4      # 彩图图视频流
```

- episode_\${idx}_qpos.png



- episode_\${idx}_base_action.png



- `episode_${idx}_video.mp4`



3.2.2.2 参数说明

- `--dataset_dir` 数据集保存路径
- `--task_name` 任务名,作为数据集的文件名
- `--episode_idx` 动作分块索引号

3.2.3 重播数据集

3.2.3.1 运行

执行这一节请关闭所有终端

- 将采集的数据集包,使用ros发布该数据包的彩色图和机械臂关节姿态
- 发布该数据包后,cobot_magic可订阅该消息,示教臂根据数据集运动

```
# 0 启动roscore
roscore

# 1 启动roscore启动从臂
roslaunch agx_arm start_all_agx_arm.launch mode:=0
rostopic pub /enable_flag std_msgs/Bool "data: true"

# 2 新启动一个终端,激活虚拟环境
conda activate aloha

# 3 发布彩色图、主臂、从臂消息
## 3.1 进入collect_data目录
cd cobot_magic/collect_data/

## 3.2 重播数据集,从臂重复执行录得数据
python replay_data.py --dataset_dir ~/data --task_name aloha_mobile_dummy --
episode_idx 0

# 4 发布彩色图、从臂消息
python replay_data.py --dataset_dir ~/data --task_name aloha_mobile_dummy
--only_pub_master --episode_idx 0
## replay_data.py运行前,请关闭遥操作代码窗口,只启动puppet.sh脚本即可
```

3.2.3.2 参数说明

- `--dataset_dir` 数据集保存路径
- `--task_name` 任务名,作为数据集的文件名
- `--episode_idx` 动作分块索引号
- `--only_pub_puppet` 是否只发布主臂的关节姿态消息

4 ACT训练推理

4.1 环境配置

- ubuntu-20.04,cuda-11.8,cudnn-8.6.0,torch-2.1.1, python-3.8已测试通过
- ubuntu-20.04,cuda-11.3,cudnn-8.6.0,torch-1.10.0,python-3.8已测试通过

```
# 1 创建conda虚拟环境
conda create -n aloha python=3.8

# 2 激活虚拟环境
conda activate aloha

# 3 安装torch==1.10.0
pip install torch==1.10.0+cu113 torchvision==0.11.0+cu113 torchaudio==0.10.0 --
extra-index-url https://download.pytorch.org/whl/cu113

# 4 安装requirements.txt
pip install -r requirements.txt

# 5 安装detr
cd aloha-devel/act/detr && pip install -v -e .
```

详情请参考[mobile-aloha](#)、[act-plus-plus](#)

4.2 数据集采集

- 参考本文3.2.1(采集数据)章节

4.3 训练

```
# 1 激活虚拟环境
conda activate aloha

# 2 训练
## 2.1 进入aloha-devel目录
cd aloha-devel

## 2.2 启动训练
python act/train.py --dataset_dir ~/data0314/ --ckpt_dir train --batch_size 4 --
num_epochs 5000 --num_episodes 50
```

1. 主要参数说明 目前仅支持ACT模型训练

- --dataset_dir 数据集目录
- --ckpt_dir 训练模型保存目录
- --batch_size 训练批量大小
- --num_epochs 训练周期
- --task_name 任务名称
- --pretrain_ckpt 预训练模型路径
- --ckpt_name 模型名称
- --num_episodes 多少组数据

4.4 推理

```
# 1 启动roscore
roscore

# 2 新启动一个终端，启动从臂与相机
## 2.1 进入remote_control目录
cd remote_control

## 2.2 启动2条从臂，启动前需要关闭机械臂遥控操作代码
roslaunch agx_arm start_all_agx_arm.launch mode:=0
rostopic pub /enable_flag std_msgs/Bool "data: true"

# 3 推理
## 3.1 激活虚拟环境
conda activate aloha

## 3.2 执行推理
## 3.2.1 进入aloha-devel目录
cd aloha-devel
## 3.2.2 推理
python inference.py --ckpt_dir train_dir
## 推理时注意安全，如果发现推理表现不正常，请立即中断代码或者断开机械臂电源，以免损伤机械臂
```

5 Q&A

Q:启动遥控操作remote.sh脚本程序, 手操作主臂, 但是主臂锁住

A: 在master1与master2终端窗口按一次i即可, 切记不要按i多次

Q: 启动remote.sh脚本时, 四条臂或者单臂无法进入初始零位位置。

A: 检查无法正常归零位的臂的电源、网口是否松动

Q: 遥控操作时,由于暴力操作或者误操作导致某条机械臂限位, 无法正常工作。

A: 正常的臂归零位状态, 无法正常运作的臂用手托着, 然后关闭机械臂所有终端或者断电, 重新启动即可

Q: 启动底盘后, 有里程计/odom消息, 但是移动车,/odom数据始终处于0数据。

A: 底盘can使能, 然后重新启动底盘程序

Q: 如何查看ros话题频率

A: 终端输入 `rostopic hz 话题名称` 即可打印该话题频率

Q: 运行collect_data.py脚本, 如果代码报错

A: 请使用cobot_magic自带的镜像环境

Q: 运行collect_data.py脚本, 如果终端一直打印 `sync fail`

A: 请检查collect_data.py收集的ros话题是否有正常输出, 没有正常输出请检查各**话题名**是否对应

Q: 终端报错信息如下:

```
RLEException: roscore cannot run as another roscore/master is already running.  
Please kill other roscore/master processes before relaunching.  
The ROS_MASTER_URI is http://PC:11311/  
The traceback for the exception was written to the log file
```

A: 由于重复启动roscore导致, 可以重启或者注销电脑

Q: 编译remote_control、camera_ws代码时。报empy,rospkg,catkin_pkg相关的错误

A: 安装empy,rospkg,catkin_pkg即可

```
pip install empy==3.3.4 catkin-pkg rospkg -i  
https://pypi.tuna.tsinghua.edu.cn/simple/
```

Q: 底盘ros里程计消息 /odom 打印,数字没有变化

A: 确定底盘 can2usb 线是否成功连接和底盘 can 是否使能

Q: 运行 ./tools/can.sh 报错 `sudo: slcand: command not found`

A: 安装一下can-utils

```
sudo apt install can-utils
```